

app\hw_mux_adg731.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      hw_mux_adg731.py
4  @brief     Pilote pour multiplexeur ADG731 via interface SPI.
5  @details   Ce module fournit un pilote pour contrôler les multiplexeurs
6             ADG731 via l'interface SPI0. Supporte jusqu'à 4 cartes avec
7             32 canaux chacune. Gère la configuration SPI et l'envoi des
8             commandes de contrôle pour la sélection des canaux.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Opérations sur les fichiers et chemins système
20 import os
21 # Interface SPI pour Linux
22 import spidev
23
24 # -----
25 # Fonctions utilitaires
26 # -----
27
28 def adg731_ctrl_byte(address, enable=True):
29     """
30     @brief    Génère le byte de contrôle pour l'ADG731.
31     @details  Construit l'octet de commande selon le protocole ADG731 avec
32               le bit d'activation (EN) en MSB et l'adresse sur 5 bits.
33               L'activation se fait avec EN=0 (logique inversée).
34
35     @param   address Adresse du canal (0-31).
36     @param   enable  État d'activation du canal (True=activé).
37
38     @return  Octet de contrôle formaté selon le protocole ADG731.
39
40     @exception ValueError Si l'adresse est hors de la plage valide.
41     """
42     # Vérification de la plage d'adresse valide
43     if address < 0 or address > 31:
44         # Lever une exception si l'adresse est invalide
45         raise ValueError("address out of range (0..31)")
46     # Logique inversée pour le bit d'activation
47     if enable is True:
48         # EN=0 pour activer (logique inversée)
49         en = 0
50     else:
51         # EN=1 pour désactiver
52         en = 1
53     # Construction de l'octet de contrôle (EN << 7) | (adresse & 0x1F)
54     ctrl = (en << 7) | (address & 0x1F)
55     return ctrl
56
57 # -----
58 # Classes principales
59 # -----
60
61 class Adg731MuxSpi:
62     """
63     @brief    Pilote SPI pour multiplexeurs ADG731.
64     @details  Classe pour contrôler jusqu'à 4 multiplexeurs ADG731 via
65               l'interface SPI0. Chaque multiplexeur peut gérer 32 canaux.
66               Configure automatiquement les paramètres SPI optimaux et
67               gère la communication avec les circuits.
68     """
69
70     # Chemins des périphériques SPI disponibles
71     DEV = ["/dev/spidev0.0", "/dev/spidev0.1", "/dev/spidev0.2", "/dev/spidev0.3"]
72
73     def __init__(self, speed_hz=100000):
74         """
75         @brief    Initialise le pilote pour les multiplexeurs ADG731.
76         @details  Configure l'interface SPI pour chaque canal disponible,
77                 définit les paramètres de communication et initialise
78                 les handles pour jusqu'à 4 cartes multiplexeurs.
79
80         @param   speed_hz Vitesse SPI en Hz (par défaut 100kHz).
81         """
82         # Stockage de la vitesse SPI configurée
83         self.speed = speed_hz
84         # Liste des handles SPI pour chaque carte

```

```

85     self.handles = []
86     # Index de boucle pour initialisation des cartes
87     index = 0
88     # Boucle d'initialisation pour les 4 cartes possibles
89     while index < 4:
90         # Récupération du chemin du périphérique SPI
91         path = self.DEV[index]
92         # Vérification de l'existence du périphérique
93         if os.path.exists(path) is True:
94             # Création d'une instance SpiDev
95             s = spidev.SpiDev()
96             # Ouverture du bus SPI (bus 0, device index)
97             s.open(0, index)
98             # Configuration du mode SPI (mode 1: CPOL=0, CPHA=1)
99             s.mode = 1
100            # Configuration de la vitesse maximale
101            s.max_speed_hz = self.speed
102            # Configuration du nombre de bits par mot
103            s.bits_per_word = 8
104            try:
105                # Configuration LSB first (False = MSB first)
106                s.lsbfirst = False
107            except Exception:
108                # Ignorer si non supporté par la plateforme
109                pass
110            try:
111                # Configuration CS high (False = CS actif bas)
112                s.cshigh = False
113            except Exception:
114                # Ignorer si non supporté par la plateforme
115                pass
116            try:
117                # Configuration no CS (False = utiliser CS)
118                s.no_cs = False
119            except Exception:
120                # Ignorer si non supporté par la plateforme
121                pass
122            try:
123                # Configuration three wire (False = mode 4 fils)
124                s.threewire = False
125            except Exception:
126                # Ignorer si non supporté par la plateforme
127                pass
128            # Ajout du handle configuré à la liste
129            self.handles.append(s)
130        else:
131            # Ajout d'un handle None si le périphérique n'existe pas
132            self.handles.append(None)
133        # Passage à l'index suivant
134        index = index + 1
135
136    def close(self):
137        """
138        @brief Ferme toutes les interfaces SPI.
139        @details Parcourt tous les handles SPI ouverts et les ferme
140        proprement. Gère les erreurs de fermeture pour éviter
141        les exceptions lors du nettoyage.
142        """
143        # Parcourt de tous les handles SPI
144        for h in self.handles:
145            # Vérification que le handle existe
146            if h is not None:
147                try:
148                    # Fermeture du handle SPI
149                    h.close()
150                except Exception:
151                    # Ignorer les erreurs de fermeture
152                    pass
153
154    def set_channel(self, board_index, address):
155        """
156        @brief Active un canal spécifique sur une carte donnée.
157        @details Envoie la commande de sélection de canal vers la carte
158        multiplexeur spécifiée. Génère l'octet de contrôle et
159        l'envoi via SPI.
160
161        @param board_index Index de la carte (0-3).
162        @param address Adresse du canal à activer (0-31).
163
164        @return Octet de contrôle envoyé, ou None si erreur.
165
166        @exception ValueError Si l'index de carte est hors plage.
167        """
168        # Vérification de la plage d'index de carte
169        if board_index < 0 or board_index > 3:
170            # Lever une exception si l'index est invalide
171            raise ValueError("board_index out of range (0..3)")

```

```
172     # Récupération du handle SPI pour la carte
173     h = self.handles[board_index]
174     # Vérification de l'existence du handle
175     if h is None:
176         print("ADG731: board", board_index, "non présent, ignoré")
177         return None
178     # Génération de l'octet de contrôle pour activer le canal
179     ctrl = adg731_ctrl_byte(address, True)
180     # Envoi de la commande via SPI
181     h.xfer2([ctrl])
182     return ctrl
183
184 def set_output_channel(self, channel_index):
185     """
186     @brief    Utilitaire simple pour sélectionner un canal de sortie.
187     @details  Fonction de commodité qui utilise automatiquement la
188               carte 0 et mappe directement l'index de canal sur
189               l'adresse du multiplexeur.
190
191     @param   channel_index Index du canal de sortie (0-31).
192
193     @return  Octet de contrôle envoyé, ou None si erreur.
194
195     @exception ValueError Si l'index de canal est hors plage.
196     """
197     # Utilitaire simple si tu veux piloter par numéro de voie unique
198     # ici: board 0, address = channel_index
199     # Vérification de la plage d'index de canal
200     if channel_index < 0 or channel_index > 31:
201         # Lever une exception si l'index est invalide
202         raise ValueError("channel_index out of range (0..31)")
203     # Appel de la méthode générique avec la carte 0
204     return self.set_channel(0, channel_index)
205
```